

Inference in First Order Logic

Inference Rules for FOL

- Inference rules for PL apply to FOL as well (Modus Ponens, And-Introduction, And-Elimination, etc.)
- New (sound) inference rules for use with **quantifiers**:
 - Universal Elimination
 - Existential Introduction
 - Existential Elimination
 - Generalized Modus Ponens (GMP)
- **Resolution**
 - Clause form (CNF in FOL)
 - Unification (consistent variable substitution)
 - Refutation resolution (proof by contradiction)

Universal Elimination $(\forall x) P(x) \vdash P(c)$.

- If $(\forall x) P(x)$ is true, then $P(c)$ is true for **any** constant c in the domain of x , i.e., $(\forall x) P(x) \models P(c)$.
- Replace all occurrences of x in the scope of $\forall x$ by the **same** ground term (a constant or a ground function).
- Example: $(\forall x) \text{eats}(\text{Ziggy}, x) \vdash \text{eats}(\text{Ziggy}, \text{IceCream})$

Existential Introduction $P(c) \vdash (\exists x) P(x)$

- If $P(c)$ is true, so is $(\exists x) P(x)$, i.e., $P(c) \models (\exists x) P(x)$
- Replace all instances of the given constant symbol by the same **new** variable symbol.
- Example $\text{eats}(\text{Ziggy}, \text{IceCream}) \vdash (\exists x) \text{eats}(\text{Ziggy}, x)$

Existential Elimination $(\exists x)P(x) \vdash P(c)$

- where c is a **new** constant symbol standing for object that makes P true,
 - All we know is there must be some constant that makes this true, so we can introduce a brand new one to stand in for that constant, *even though we don't know exactly what that constant refer to.*
 - Example: $(\exists x) \text{eats}(\text{Ziggy}, x) \models \text{eats}(\text{Ziggy}, \text{Stuff})$

- Things become more complicated when there are universal quantifiers

$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash (\forall x)\text{eats}(x, \text{Stuff}) ???$

$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash \text{eats}(\text{Ziggy}, \text{Stuff}) ???$

- Introduce a **new** function *food*(x) to stand for $\exists y$ because that y depends on x

$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash (\forall x)\text{eats}(x, \text{food}(x))$

$(\forall x)(\exists y) \text{ eats}(x, y) \dashv\vdash \text{eats}(\text{Ziggy}, \text{food}(\text{Ziggy}))$

- What exactly the function *food*(.) does is unknown, except that it takes x as its argument

- The process of existential elimination is called **Skolemization** (after Skolem, a Norwegian mathematician). The new, unique constants (e.g., Stuff) and functions (e.g., food(.)) are called skolem constants and skolem functions

Generalized Modus Ponens (GMP)

- Combines And-Introduction, Universal-Elimination, and Modus Ponens

- Ex: $\mathbf{P(c), Q(c), (\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x) \vdash R(c)}$

$P(c), Q(c) \vdash P(c) \wedge Q(c)$ *(by and-introduction)*

$(\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x)$

$\vdash (P(c) \wedge Q(c)) \Rightarrow R(c)$ *(by universal-elimination)*

$P(c) \wedge Q(c), (P(c) \wedge Q(c)) \Rightarrow R(c) \vdash R(c)$ *(by modus ponens)*

Resolution for FOL

- **Resolution** is a rule of inference used in logic, particularly in propositional and first-order logic, to derive conclusions from a set of premises.

Key Concepts of Resolution:

1. Literals and Clauses:

1. A **literal** is an atomic proposition or its negation. For example, $P(x)$ (positive literal) or $\neg P(x)$ (negative literal).
2. A **clause** is a disjunction (OR) of literals. In propositional logic, a clause might look like $P \vee Q \vee \neg R$. In first-order logic (FOL), the literals can involve variables and quantifiers.

2. Opposite Literals:

In resolution, the rule applies when two clauses contain **complementary literals**. These are literals that cancel each other out because one is positive, and the other is negative, like $P(x)$ and $\neg P(x)$.

3. The Resolution Rule:

The **resolution rule** states that if two clauses contain opposite literals, a new clause (called the **resolvent**) can be derived by removing the opposite literals and combining the remaining ones from both clauses.

1. For example, if you have:

1. Clause 1: $P(x) \vee Q(x)$

2. Clause 2: $\neg P(x) \vee R(x)$

2. By **resolving** $P(x)$ and $\neg P(x)$, you derive a new clause:

Resolvent: $Q(x) \vee R(x)$

- **Unification** (in First-Order Logic):
- In first-order logic, variables can be present, so unification is required before applying resolution. Unification is the process of finding a substitution that makes two literals identical. For example, to resolve $P(x)$ and $\neg P(y)$, the variables x and y need to be unified by some substitution, such as $x=y$

We need answers to the following questions

- How to convert FOL sentences to clause form (especially how to remove quantifiers)
- How to unify two argument lists.
- How to determine which two clauses in KB should be resolved next (among all resolvable pairs of clauses) and how to determine a proof is completed

Conversion Procedure of FOL sentences to clause form

step 1: remove all “ $\Rightarrow$ ” and “ $\Leftrightarrow$ ” operators

(using $P \Rightarrow Q \equiv \sim P \vee Q$ and $P \Leftrightarrow Q \equiv P \Rightarrow Q \wedge Q \Rightarrow P$)

step 2: move all negation signs to individual predicates

(using de Morgan’s law)

step 3: remove all existential quantifiers $\exists y$

case 1: y is not in the scope of any universally quantified variable,
then replace all occurrences of y by a **skolem constant**

case 2: if y is in scope of universally quantified variables $x_1, \dots, x_i$,
then replace all occurrences of y by a **skolem function** that
takes $x_1, \dots, x_i$ as its arguments

step 4: remove all universal quantifiers $\forall x$ (with the understanding that
all remaining variables are universally quantified)

step 5: convert the sentence into CNF (using distribution law, etc)

step 6: use parenthesis to separate all disjunctions, then drop all $\vee$ ’s and
 $\wedge$ ’s

Conversion examples

$\forall x (P(x) \wedge Q(x) \Rightarrow R(x))$

$\forall x \sim(P(x) \wedge Q(x)) \vee R(x)$ (by step 1)

$\forall x \sim P(x) \vee \sim Q(x) \vee R(x)$ (by step 2)

$\sim P(x) \vee \sim Q(x) \vee R(x)$ (by step 4)

$(\sim P(x), \sim Q(x), R(x))$ (by step 6)

$\exists y \text{ rose}(y) \wedge \text{yellow}(y)$

$\text{rose}(c) \wedge \text{yellow}(c)$

(where c is a skolem constant)

$(\text{rose}(c), \text{yellow}(c))$

$\forall x [\text{person}(x) \Rightarrow \exists y (\text{person}(y) \wedge \text{father}(y, x))]$

$\forall x [\sim \text{person}(x) \vee \exists y (\text{person}(y) \wedge \text{father}(y, x))]$ (by step 1)

$\forall x [\sim \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))]$ (by step 3)

$\sim \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))$ (by step 4)

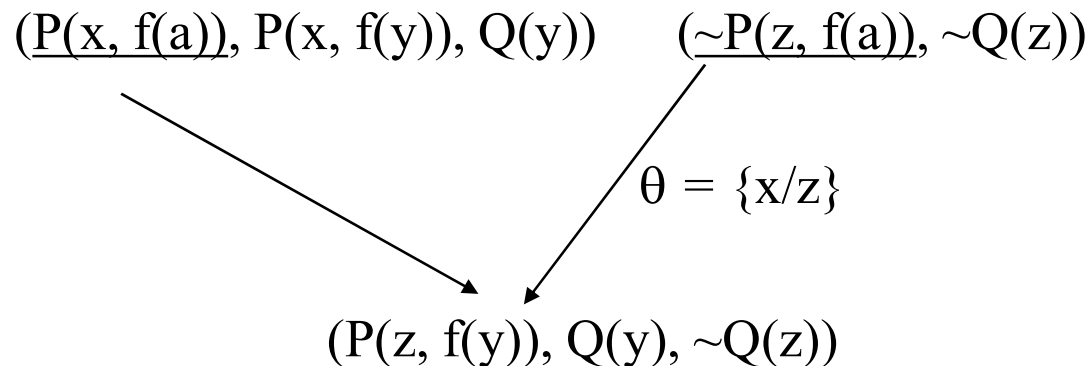
$(\sim \text{person}(x) \vee \text{person}(f_sk(x))) \wedge (\sim \text{person}(x) \vee \text{father}(f_sk(x), x))$ (by step 5)

$(\sim \text{person}(x), \text{person}(f_sk(x))), (\sim \text{person}(x), \text{father}(f_sk(x), x))$ (by step 6)

(where $f_sk(.)$ is a skolem function)

Resolution in FOL

- Convert all sentences in KB (axioms, definitions, and known facts) and the goal sentence (the theorem to be proved) to clause form
- Two clauses $C1$ and $C2$ can be resolved if and only if r in $C1$ and s in $C2$ are two opposite literals, and their argument list $arglist_r$ and $arglist_s$ are unifiable with most general unifier (MGU) $= \theta$.
- Then derive the resolvent sentence: $subst((C1 - \{r\}, C2 - \{s\}), \theta)$
(substitution is applied to all literals in $C1$ and $C2$, but not to any other clauses)
- Example

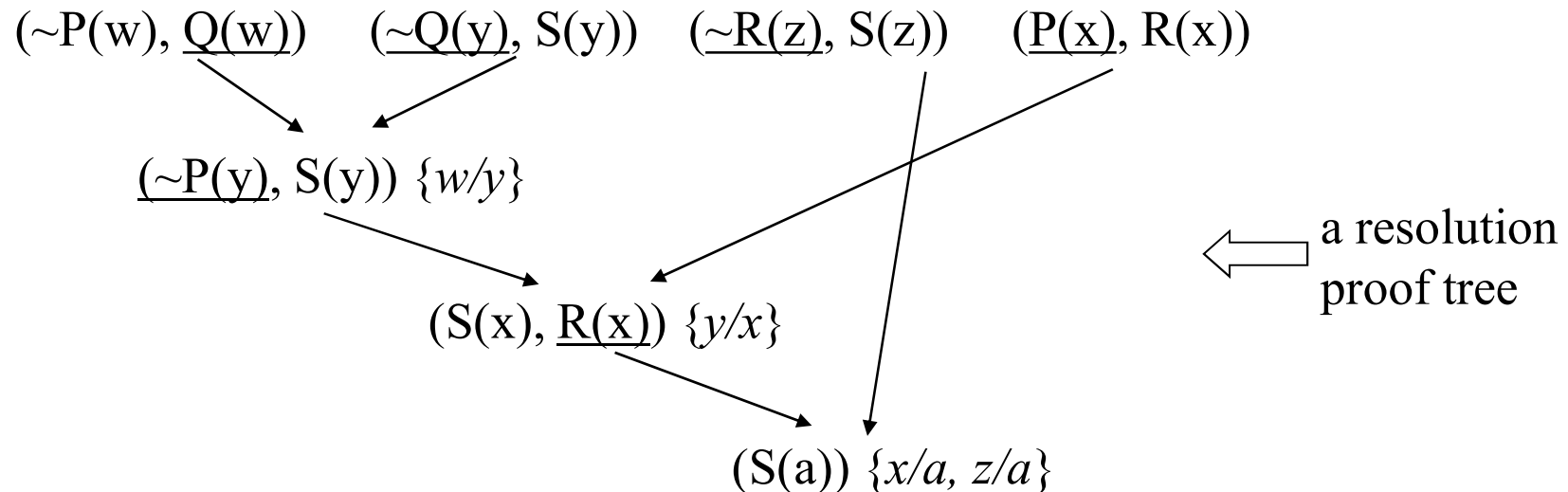


Resolution example

- Prove that

$\forall w P(w) \Rightarrow Q(w), \forall y Q(y) \Rightarrow S(y), \forall z R(z) \Rightarrow S(z), \forall x P(x) \vee R(x) \models \exists u S(u)$

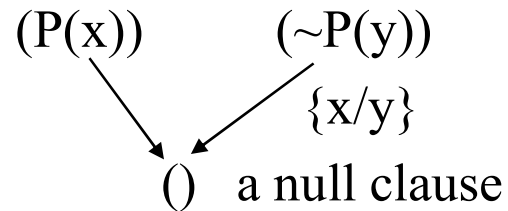
- Convert these sentences to clauses ($\exists u S(u)$ skolemized to $S(a)$)
- Apply resolution



- Problems
 - The theorem $S(a)$ does not actively participate in the proof
 - Hard to determine if a proof (with consistent variable bindings) is completed if the theorem consists of more than one clause

Resolution Refutation: a better proof strategy

- Given a consistent set of axioms KB and goal sentence Q, show that $KB \models Q$.
- **Proof by contradiction:** Add $\sim Q$ to KB and try to prove false.
because $(KB \models Q) \iff (KB \wedge \sim Q \models \text{False}, \text{ or } KB \wedge \sim Q \text{ is inconsistent})$
- How to represent “**false**” in clause form
 - $P(x) \wedge \sim P(y)$ is inconsistent
 - Convert them to clause form then apply resolution

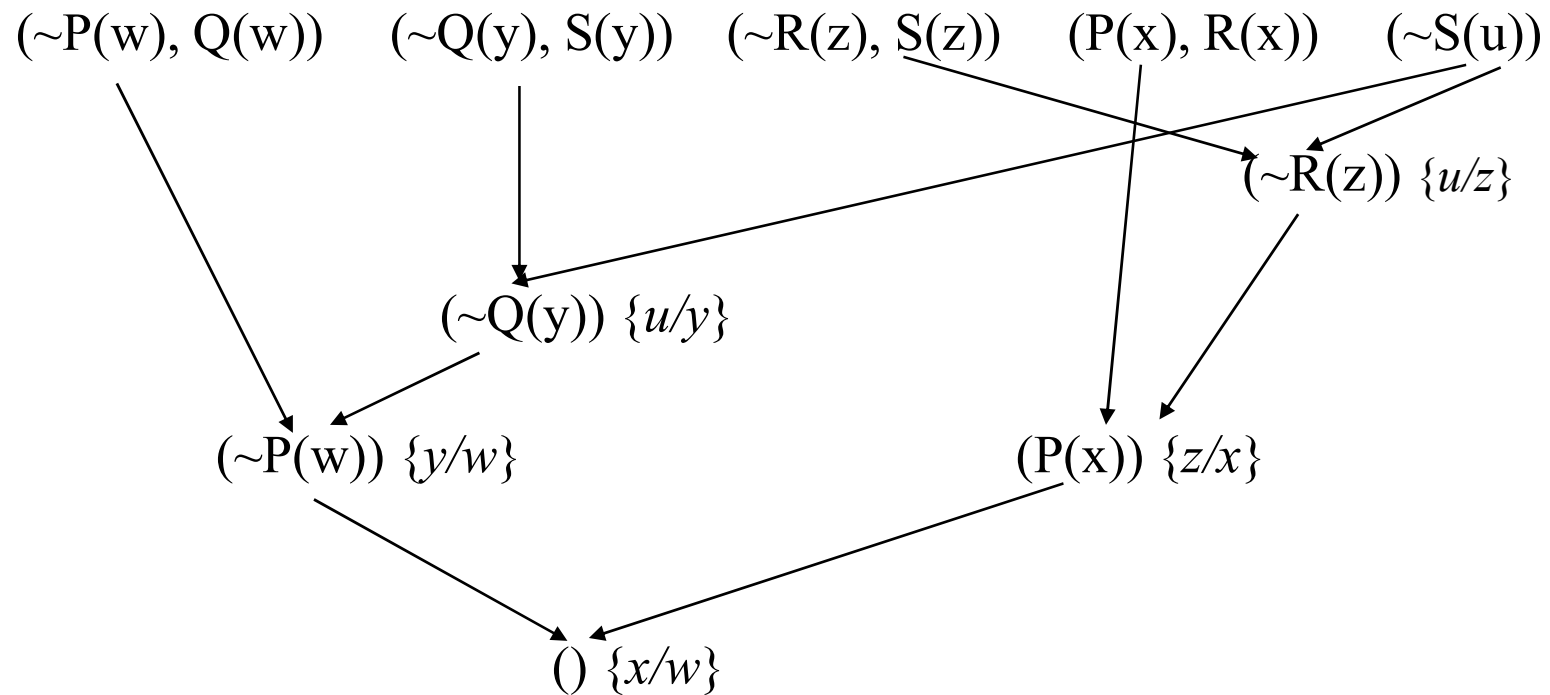


- A null clause represents false (inconsistence/contradiction)
- **$KB \models Q$ if we can derive a null clause from $KB \wedge \sim Q$ by resolution**

- Prove by resolution refutation that

$$\forall w P(w) \Rightarrow Q(w), \forall y Q(y) \Rightarrow S(y), \forall z R(z) \Rightarrow S(z), \forall x P(x) \vee R(x) \models \exists u S(u)$$

- Convert these sentences to clauses ($\sim \exists u S(u)$ becomes $\sim S(u)$)



Example of Automatic Theorem Proof:

Did Curiosity kill the cat

- Jack owns a dog. Every dog owner is an animal lover. No animal lover kills an animal. Either Jack or Curiosity killed the cat, who is named Tuna. Did Curiosity kill the cat?
- These can be represented as follows:
 - A. $(\exists x) \text{Dog}(x) \wedge \text{Owns}(\text{Jack}, x)$
 - B. $(\forall x) ((\exists y) \text{Dog}(y) \wedge \text{Owns}(x, y)) \Rightarrow \text{AnimalLover}(x)$
 - C. $(\forall x) \text{AnimalLover}(x) \Rightarrow (\forall y) \text{Animal}(y) \Rightarrow \sim \text{Kills}(x, y)$
 - D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$
 - E. $\text{Cat}(\text{Tuna})$
 - F. $(\forall x) \text{Cat}(x) \Rightarrow \text{Animal}(x)$
 - Q. $\text{Kills}(\text{Curiosity}, \text{Tuna})$

- **Convert to clause form**

A1. (Dog(D)) /* D is a skolem constant */

A2. (Owns(Jack,D))

B. ($\sim$ Dog(y), $\sim$ Owns(x, y), AnimalLover(x))

C. ($\sim$ AnimalLover(x), $\sim$ Animal(y), $\sim$ Kills(x,y))

D. (Kills(Jack,Tuna), Kills(Curiosity,Tuna))

E. (Cat(Tuna))

F. ($\sim$ Cat(x), Animal(x))

- **Add the negation of query:**

$\neg$ Q: ($\sim$ Kills(Curiosity, Tuna))

- **The resolution refutation proof**

R1: $\neg Q$, D, {}, (Kills(Jack, Tuna))

R2: R1, C, {x/Jack, y/Tuna}, ($\sim$ AnimalLover(Jack), $\sim$ Animal(Tuna))

R3: R2, B, {x/Jack}, ($\sim$ Dog(y), $\sim$ Owns(Jack, y), $\sim$ Animal(Tuna))

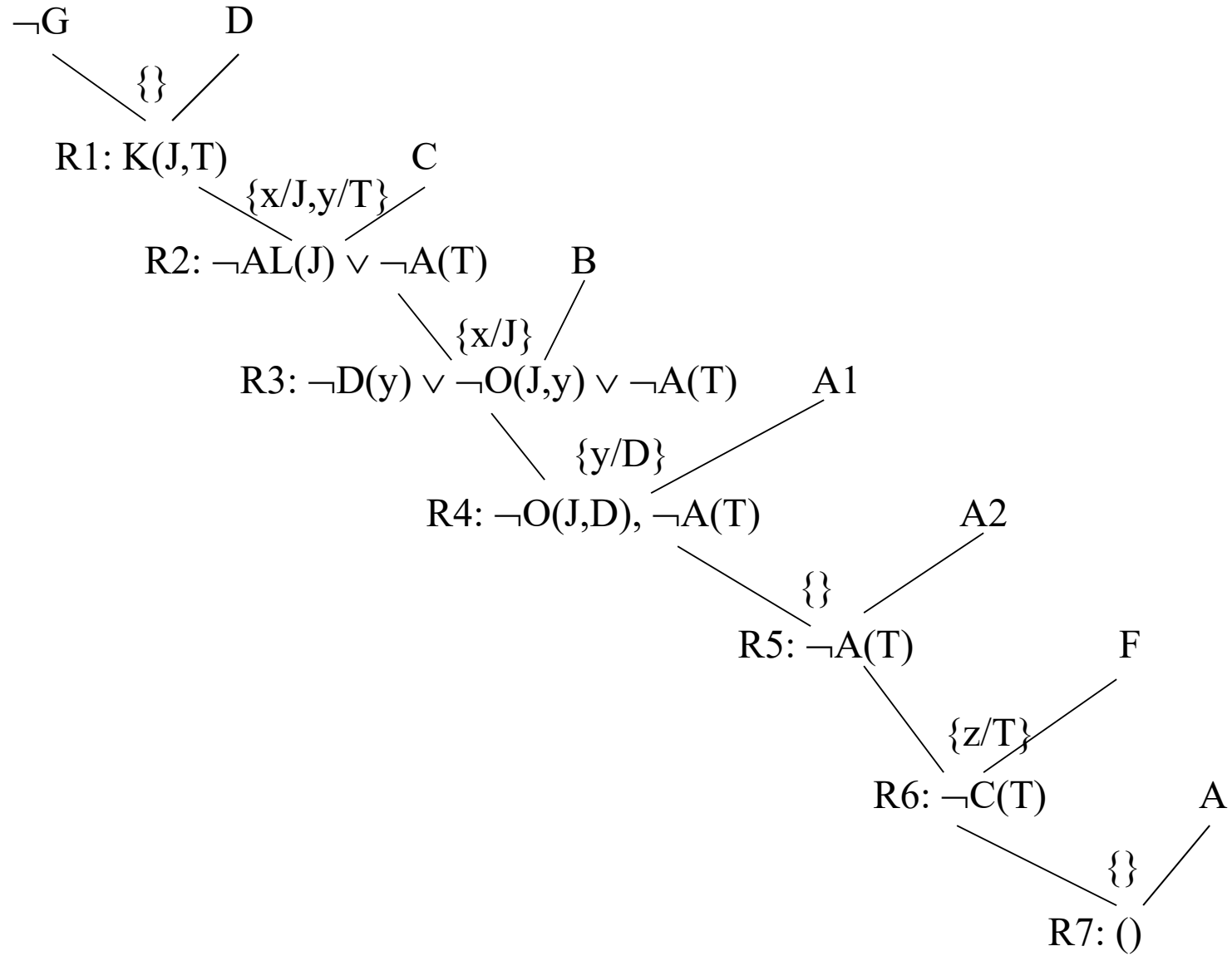
R4: R3, A1, {y/D}, ($\sim$ Owns(Jack, D), $\sim$ Animal(Tuna))

R5: R4, A2, {}, ($\sim$ Animal(Tuna))

R6: R5, F, {x/Tuna}, ($\sim$ Cat(Tuna))

R7: R6, E, {} ()

- **The proof tree**



- Forward chaining and Backward chaining
 - Implication rules are always used in the way of **modus ponens (from premises to conclusions)**, i.e., in the direction of implication arrows.
 - This defines a forward chaining inference procedure because it moves "forward" from fact toward the goal (also called **data driven**).
 - Implication rules are always used in the way of modus tollens (*from conclusions to premises*), i.e., in the reverse direction of implication arrows
 - This defines a backward chaining inference procedure because it moves "backward" from the goal (also called **goal driven**).

- Proof starts with the goal query (theorem to be proven) $\Leftarrow Q(a)$
- Resolve it with rules $Q(x) \Leftarrow P(x)$ to derived new query $\Leftarrow P(a)$
- Additional inference is then triggered by $\Leftarrow P(a)$, etc. The process stops when a null clause is derived.
- Backward chaining is more efficient than forward chaining as it is more focused. However, it requires that the goal (theorem to be proven) be known prior to the inference
- Avoid loops and repeated work: check if new subgoal
 - is already on the goal stack
 - has already been proved true or failed